

Angular Service-ek és HTTP kommunikáció

A kliens-szerver összekötés elméleti alapjai

Összefoglaló tananyag
Programrendszerek fejlesztése gyakorlat
2026. tavasz

1. Bevezetés – Kontextus és motiváció

A félév során felépítettünk egy teljes backend rendszert: az **Express.js** szerver **REST API**-kat szolgáltat, amelyek **MongoDB** adatbázissal kommunikálnak. Megvalósítottuk az autentikációt (JWT), a jogosultságkezelést és az alapvető CRUD műveleteket. Az Angular alkalmazásunk felhasználói felülete szintén kész, de jelenleg statikus vagy mock adatokkal dolgozik.

A mai tananyag célja, hogy életre keltsük a frontend-et: összekössük a kész UI-t a működő backend-del. Ehhez az Angular **service réteget** használjuk, amely a modern frontend architektúrák egyik legfontosabb építőeleme.

Tanulási cél

A fejezet végére a hallgatók megértik, miért van szükség service rétegre, hogyan épül fel egy Angular service, és hogyan valósítható meg rajta keresztül a teljes kliens-szerver kommunikáció.

2. Miért van szükség service rétegre?

2.1. A probléma: összemosott felelősségek

Ha a HTTP hívásokat közvetlenül a komponensbe írjuk, az összemossa a **megjelenítési** és az **adathozzáférési** logikát. A komponens egyszerre lesz felelős a DOM kezeléséért és a hálózati kommunikációért, ami több problémát is okoz:

- **Tesztelhetőség:** a komponens teszteléséhez valós HTTP hívásokat (vagy azok mockolását) kell kezelni.
- **Újrafelhasználhatóság:** ha másik komponensnek is kell ugyanaz az adatforrás, a logikát másolni kell.
- **Karbantarthatóság:** egy API végpont változása esetén minden érintett komponensben módosítani kell.

2.2. A megoldás: Single Responsibility Principle (SRP)

A **Single Responsibility Principle** (egyetlen felelősség elve) kimondja, hogy egy osztálynak egyetlen oka legyen a változásra. A frontend kontextusában ez azt jelenti:

- **Komponens** = megjelenítés és felhasználói interakciók kezelése
- **Service** = adatforrás-kezelés, üzleti logika, állapotkezelés

A service réteg bevezetésével a komponens nem tudja és nem is kell tudnia, honnan jön az adat. Ez lehet REST API, WebSocket, lokális cache vagy akár mock adat – a komponens szempontjából ez transzparens. Ezt az elvet nevezi a szoftvertervezés **separation of concerns**-nek (felelősségek szétválasztása).

Gondolkodj úgy

A service olyan, mint egy 'adatszolgáltató iroda': a komponens beállít és kér adatot, de nem érdekli, hogy az iroda telefonál, e-mailt ír vagy személyesen megy el az információért.

3. Az Angular Service felépítése

3.1. Az @Injectable() dekorátor

Angularban a service egy egyszerű TypeScript osztály, amelyet az `@Injectable()` dekorátor jelöl meg. Ez a dekorátor jelzi az Angular **Dependency Injection (DI)** rendszerének, hogy ez az osztály injectálható más osztályokba.

```
@Injectable({
  providedIn: 'root'
})
export class RecipeService {
  // service logika
}
```

3.2. A providedIn: 'root' jelentése

A `providedIn: 'root'` beállítás azt jelenti, hogy az Angular egyetlen példányt (**singleton**) hoz létre a service-ből, és ez a példány az egész alkalmazásban elérhető. Ez két fontos következménnyel jár:

- **Konzisztens állapot:** ha a service tárol adatot (pl. auth token, cache-elt lista), az minden komponensben ugyanaz lesz.
- **Tree-shakeable:** ha egyetlen komponens sem használja a service-t, a build folyamat automatikusan kihagyja a végső bundle-ből.

3.3. Dependency Injection a gyakorlatban

A service-t a komponens konstruktorán keresztül injectáljuk. Az Angular DI rendszere automatikusan létrehozza (vagy újrafelhasználja) a példányt:

```
export class RecipeListComponent {
  constructor(private recipeService: RecipeService) {}

  ngOnInit() {
    this.recipeService.getRecipes().subscribe(data => {
      this.recipes = data;
    });
  }
}
```

A `private` kulcsszó a TypeScript rövidítése: egyszerre deklarálja és inicializálja az osztály mezőjét.

3.4. Létrehozás CLI-vel

Az Angular CLI automatikusan létrehozza a service vázát a megfelelő struktúrában:

```
ng generate service services/recipe
```

Ez létrehozza a `src/app/services/recipe.service.ts` fájlt a megfelelő dekorátorral és alap struktúrával.

4. Az HttpClient modul

4.1. Bevezetés és konfiguráció

Az Angular beépített HTTP kliense a `HttpClient` osztály, amely a `@angular/common/http` csomagban található. Standalone alkalmazásokban (Angular 17+) a konfiguráció az `app.config.ts`-ben történik:

```
// app.config.ts
import { provideHttpClient } from '@angular/common/http';

export const appConfig = {
  providers: [
    provideHttpClient()
  ]
};
```

4.2. Observable-alapú működés

Az `HttpClient` minden HTTP hívása egy **Observable**-t ad vissza, nem Promise-t. Ez az RxJS könyvtár alapfogalma, és több előnnyel jár:

- **Lemondható (unsubscribe):** ha a felhasználó elnavigál az oldalról, a folyamatban lévő kérést lemondhatjuk, megelőzve a felesleges hálózati forgalmat és a memóriaszivárgást.
- **Operátorokkal transzformálható:** az RxJS operátorok (map, filter, catchError, retry) segítségével az adatfolyamot a subscribe előtt alakíthatjuk.
- **Több érték kibocsátása:** bár HTTP kéréseknél általában egyetlen válasz érkezik, az Observable mintegy készleg a bonyolultabb forgatókönyvekre (pl. Server-Sent Events).

Observable vs Promise – mikor melyik?

A Promise egyszer teljesül és nem mondható le. Az Observable egy adatfolyam, amely több értéket bocsáthat ki idővel, és bármikor leiratkozhatunk róla. HTTP kéréseknél a lemondhatóság és az operátorok a fő előnyök.

5. A CRUD service mintázat

5.1. Tipikus metódusok

Egy domain-specifikus service (pl. `RecipeService`) tipikusan öt fő metódust tartalmaz, amelyek egy-egy HTTP metódusra képeződnek le:

Service metódus	HTTP metódus	Leírás
<code>getAll()</code>	GET	Az összes elem lekérése
<code>getId(id)</code>	GET <code>/:id</code>	Egy elem lekérése azonosító alapján
<code>create(data)</code>	POST	Új elem létrehozása
<code>update(id, data)</code>	PUT / PATCH	Meglévő elem módosítása
<code>delete(id)</code>	DELETE	Elem törlése

5.2. Példa: `RecipeService`

```
@Injectable({ providedIn: 'root' })
export class RecipeService {
  private apiUrl = environment.apiUrl + '/recipes';

  constructor(private http: HttpClient) {}

  getRecipes(): Observable<Recipe[]> {
    return this.http.get<Recipe[]>(this.apiUrl);
  }

  getRecipeById(id: string): Observable<Recipe> {
    return this.http.get<Recipe>(`${this.apiUrl}/${id}`);
  }

  createRecipe(data: Recipe): Observable<Recipe> {
    return this.http.post<Recipe>(this.apiUrl, data);
  }

  updateRecipe(id: string, data: Recipe): Observable<Recipe> {
    return this.http.put<Recipe>(`${this.apiUrl}/${id}`, data);
  }

  deleteRecipe(id: string): Observable<void> {
    return this.http.delete<void>(`${this.apiUrl}/${id}`);
  }
}
```

5.3. Az environment konfiguráció

Az API alap URL-jét soha nem szórjuk szét a service-ekben. Ehelyett az `environment.ts` fájlban definiáljuk, és onnan importáljuk. Ez lehetővé teszi, hogy fejlesztési és éles környezet között egyetlen helyen váltsunk:

```
// environment.ts (fejlesztés)
export const environment = {
  production: false,
  apiUrl: 'http://localhost:3000/api'
};

// environment.prod.ts (éles)
export const environment = {
  production: true,
  apiUrl: 'https://myapp.example.com/api'
};
```

6. Autentikáció kezelése service szinten

6.1. Az AuthService felelőssége

Az **AuthService** központosítja az autentikációval kapcsolatos összes logikát: bejelentkezés, regisztráció, kijelentkezés, token tárolás és a bejelentkezési állapot lekérdezése. A JWT tokenet a **localStorage**-ban tároljuk, és minden autentikált kéréshez csatolnunk kell.

```
@Injectable({ providedIn: 'root' })
export class AuthService {
  private apiUrl = environment.apiUrl + '/auth';

  constructor(private http: HttpClient) {}

  login(credentials: {email: string, password: string}) {
    return this.http.post<{token: string}>(
      `${this.apiUrl}/login`, credentials
    ).pipe(
      tap(res => localStorage.setItem('token', res.token))
    );
  }

  getToken(): string | null {
    return localStorage.getItem('token');
  }

  isLoggedIn(): boolean {
    return !!this.getToken();
  }

  logout(): void {
    localStorage.removeItem('token');
  }
}
```

6.2. A token csatolásának két megközelítése

Két módszer létezik a JWT token kérésekhez való csatolására:

- **Manuális:** minden service-ben kézzel hozzáadjuk az Authorization headert minden egyes kéréshez. Ez ismétlődő és hibaérzékeny.
- **Interceptor (ajánlott):** egyetlen központi ponton automatikusan hozzáfűzzük a token az összes kimenő kéréshez. Ez a modern Angular best practice.

7. HttpInterceptor – cross-cutting concerns

7.1. Mi az Interceptor?

Az **HttpInterceptor** egy kliens oldali middleware: elkapja az összes kimenő HTTP kérést (vagy bejövő választ), és módosíthatja azokat, mielőtt tovább haladnak. Klasszikus felhasználási területek:

- Autentikációs token automatikus csatolása minden kéréshez
- Globális hibakezelés (pl. 401 → átirányítás a login oldalra)
- Loading indikátor automatikus megjelenítése/elrejtése
- Loggolás, caching, retry logika

7.2. Functional Interceptor (Angular 17+)

A modern Angular a **functional interceptor** szintaxist használja, amely egyszerűbb és tömörebb a korábbi osztály-alapú megoldásnál:

```
// auth.interceptor.ts
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const authService = inject(AuthService);
  const token = authService.getToken();

  if (token) {
    const clonedReq = req.clone({
      setHeaders: { Authorization: `Bearer ${token}` }
    });
    return next(clonedReq);
  }

  return next(req);
};
```

7.3. Interceptor regisztráció

Az interceptort az **app.config.ts**-ben regisztráljuk a **withInterceptors()** függvénnyel:

```
// app.config.ts
import { provideHttpClient, withInterceptors } from '@angular/common/http';
```

```
export const appConfig = {
  providers: [
    provideHttpClient(withInterceptors([authInterceptor]))
  ]
};
```

Fontos

Az interceptor a kérés klonozásával (`req.clone()`) működik, mert az Angular HTTP kérés objektumai immutábilisak (nem módosíthatók közvetlenül).

8. Hibakezelési stratégiák

8.1. Service-szintű hibakezelés

Az RxJS `catchError` operátora lehetővé teszi, hogy a HTTP hibákat a service-ben kapjuk el és alakítsuk át felhasználóbarát formátumba:

```
getRecipes(): Observable<Recipe[]> {
  return this.http.get<Recipe[]>(this.apiUrl).pipe(
    catchError(error => {
      console.error('Hiba a receptek betöltésekor:', error);
      return throwError(() => new Error('Nem sikerült betölteni.'));
    })
  );
}
```

8.2. HTTP státuszkódok és kezelésük

Kód	Jelentés	Javasolt kezelés
401	Unauthorized	Átírányítás a login oldalra, token törlése
403	Forbidden	"Nincs jogosultság" üzenet megjelenítése
404	Not Found	"Nem található" üzenet, vissza a listára
500	Server Error	Általános hibaüzenet, újrapróbálás felkínálása

8.3. Globális vs. lokális hibakezelés

A két megközelítés kombinálható: az **interceptor** kezeli az autentikációs hibákat (401, 403) globálisan, míg az egyes **service-ek** a domain-specifikus hibákat kezelik lokálisan (pl. "Ez a recept már nem létezik"). Ez a rétegzett megközelítés biztosítja, hogy minden hiba megfelelően legyen kezelve, anélkül, hogy bármelyik réteg túl sok felelősséget viseljen.

9. Komponens és service együttműködése

9.1. A subscribe() minta

A komponens a service-től kapott Observable-re a `subscribe()` metódussal iratkozik fel. A feliratkozás a komponensben történik, és itt kezeljük a sikeres és hibás válaszokat is:

```
ngOnInit() {
  this.recipeService.getRecipes().subscribe({
    next: (data) => this.recipes = data,
    error: (err) => this.errorMessage = err.message
  });
}
```

9.2. Az AsyncPipe alternatíva

Az `AsyncPipe` lehetővé teszi, hogy a template-ben közvetlenül Observable-re irakozzunk fel, és az Angular automatikusan leiratkozik a komponens megsemmisülésekor. Ez a megközelítés csökkenti a memóriaszivárgás kockázatát:

```
// Komponens
recipes$ = this.recipeService.getRecipes();

// Template
@for (recipe of recipes$ | async; track recipe._id) {
  <app-recipe-card [recipe]="recipe" />
}
```

Best practice

Ha a komponens csak megjeleníti az adatot és nem manipulálja, az `AsyncPipe` az ajánlott megoldás. Ha az adatot a komponens logikájában is használni kell (pl. szűrés, rendezés), a `subscribe()` rugalmasabb.

9.3. Leiratkozás fontossága

Ha `subscribe()`-ot használunk, a leiratkozásról gondoskodnunk kell, különben memóriaszivárgás léphet fel. HTTP kéréseknél ez kevésbé kritikus (mert a kérés befejezése után az Observable automatikusan lezárul), de hosszú életű Observable-öknél (pl. WebSocket, interval) elengedhetetlen. Lehetőségek: `takeUntilDestroyed()` operátor (Angular 16+), manuális leiratkozás az `ngOnDestroy()`-ban, vagy az említett `AsyncPipe`.

10. Best practices összefoglaló

Az alábbi elvek követése biztosítja a karbantartható, skálázható és tesztelhető service architektúrát:

1. **Egy service = egy domain terület.** A `RecipeService` a receptekhez, az `AuthService` az autentikációhoz, a `CategoryService` a kategóriákhoz tartozó logikát tartalmazza.

2. **A service kezeli az URL-eket és a transzformációkat.** A komponens soha nem lát nyers URL-eket vagy HTTP részleteket.
3. **Interceptor a cross-cutting logikához.** Auth token csatolás, globális hibakezelés, loggolás – ezek nem a service-ek felelőssége.
4. **Environment-ből olvasunk konfigurációt.** Soha ne hardcode-oljunk URL-eket a service-ekbe.
5. **Mindig gondoskodjunk a leiratkozásról.** Használjunk AsyncPipe-ot vagy takeUntilDestroyed()-t a memóriaszivárgás megelőzésére.
6. **Komponensek csak feliratkoznak és megjelenítenek.** Ne végezzenek üzleti logikát vagy adatmanipulációt, amit a service-nek kellene.

11. Architektúra áttekintés

Az alábbi összefoglalás bemutatja, hogyan áramlanak az adatok a rendszer rétegei között:

Komponens	A felhasználó látja és interakcióba lép vele. Service-t hív, eredményt megjelenít.
↓ hívás	A komponens a service metódusát hívja (pl. getRecipes())
Service	HTTP kérést küld az HttpClient-en keresztül. Observable-t ad vissza.
↓ HTTP	A kérés áthalad az Interceptoron (token csatolás).
Interceptor	Módosítja a kérést (Authorization header) és/vagy a választ.
↓ hálózat	A kérés eljut a REST API-hoz.
Express.js	Feldolgozza a kérést, lekérdezi a MongoDB-t, válaszol.

Ez a dokumentum a Programrendszerek fejlesztése kurzus 11. gyakorlatának elméleti összefoglalója.

MEAN Stack – 2026. tavasz